

DBMS Assignment 3

Date: _____

$$Q1) \pi_{\text{Platform-Name}} (\text{PLATFORM} \bowtie \pi_{\text{Platform-ID}} (\sigma_{\text{Difficulty-Level} = \text{"Advanced"}} (\text{COURSE}) \bowtie \text{ENROLLMENT}))$$

explanation:- filter courses with advanced difficulty, join with ENROLLMENT to get Platform-ID's then joint with Platform to retrieve platform names.

$$Q2) \pi_{\text{INSTRUCTOR-NAME}} (\text{INSTRUCTOR} \bowtie \pi_{\text{Instructor-ID}} (\sigma_{\text{status} = \text{"completed"}} (\text{COURSE-ACTIVITY}) \bowtie$$

$$\pi_{\text{Enroll-ID}} (\text{ENROLLMENT} \bowtie \sigma_{\text{category} = \text{"Data Science"}} (\text{COURSE})))$$

explanation:- first identifies Data Science courses, find the associated enrollments, filters for completed activities to isolate the correct Instructor-ID and finally retrieve the instructor name.

$$3) \pi_{\text{Platform-ID}} (\text{PLATFORM}) - \pi_{\text{Platform-ID}} (\text{INSTRUCTOR})$$

explanation:- Uses the set difference operator to find Platform IDs that exist in the platform registry. (Subtract the set of platform-ID that appears in INSTRUCTOR from all platform-ID in PLATFORM, leaving only platforms with no instructors.

$$4) E1 \leftarrow \rho_{E1} (\text{ENROLLMENT})$$

$$E2 \leftarrow \rho_{E2} (\text{ENROLLMENT})$$

$$\pi_{E1.\text{Course-ID}} (\sigma_{E1.\text{Course-ID} = E2.\text{Course-ID} \wedge E1.\text{Platform-ID} \neq E2.\text{Platform-ID} (E1 \times E2))$$

explanation:- Self-join enrollment on same course-ID but different platform-ID to find courses appearing on more than one platform.

5) $\pi (\sigma_{\text{category} = "AI" \wedge \text{Completion-Rate} > 85} (\text{COURSE} \bowtie \text{ENROLLMENT} \bowtie \text{PLATFORM}))$
 Course-Name, Platform-Name, Completion-Rate

explanation:- Join Course, enrollment and platform, then filter for AI category and completion rate above 85, projecting the required columns.

Question 2

i) Meaning:- Retrieve the titles of all books that are currently not available (checked out) AND have a price greater than 1000.

```
SELECT Title FROM Book
WHERE IsAvailable = 0 AND price > 1000;
```

ii) Meaning:- Retrieve titles of books that are either Fiction with price above 800, OR belong to the History genre (regardless of price).

```
SELECT Title FROM Book
WHERE (genre = 'Fiction' AND price > 800) OR genre = 'History';
```

iii) Meaning :- Retrieve the title and price of all Science books that are currently available.

```
SELECT title, price  
FROM Book  
WHERE genre = 'Science' AND isAvailable = 1;
```

iv) Meaning :- Retrieve the title and 1/10 of the price for all available Science books.

```
SELECT title, price / 10 As discounted-price FROM Book  
WHERE genre = 'Science' AND isAvailable = 1;
```

v) Meaning :- Find the publisher ID's of all publishers who have published the book titled AI basics.

```
SELECT p.publisherID FROM Publishes p  
JOIN Book b ON p.bookID = b.bookID  
WHERE b.title = 'AI Basics';
```

vi) Meaning :- Retrieve the staff name and library name for staff members whose home address is in the same city as the library they work in.

```
SELECT s.name, l.libName FROM Staff s  
JOIN Library l ON s.libID = l.libID  
WHERE s.address = l.city;
```

vii) Meaning:- Retrieve Staff IDs of all staff members whose salary is not equal to 60000.

```
SELECT staffID FROM Staff
WHERE salary <> 60000;
```

viii) Meaning:- Find publisher who have offices in both Karachi and Islamabad (Intersection of Publisher IDs from both cities)

```
SELECT publisherID FROM Publisher WHERE city = 'Karachi'
INTERSECT
SELECT publisherID FROM Publisher WHERE city = 'Islamabad';
```

ix) Meaning:- Retrieve title of books that have no publisher.

```
SELECT title FROM Book
WHERE bookID NOT IN (
    SELECT bookID FROM Publishers
);
```

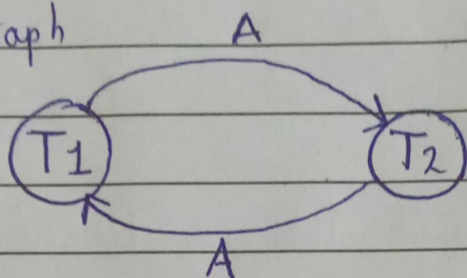
①3a) Non-Serial Schedule

Step	T1	T2
1	read(A)	
2		read(A)
3	$A := A - N$	
4		$A := A + M$
5	write(A)	
6		write(A)

This is non-serial because T_1 and T_2 are interleaved - T_2 reads A before T_1 even write it.

ii) Conflicting operation occurring during this transaction is
Read $\rightarrow$ Write, and Write - Write.

Precedence Graph



edge $T_1 \rightarrow T_2$

$R_1(A) = W_2(A)$

$R_2(A) \rightarrow W_1(A)$

step 5 and step 6

$W_1(A) - W_2(A)$

iii) Since the precedence graph contains a cycle, the schedule is NOT conflict serializable.

iv)	Step	T_1	T_2
	1	read A	
	2	$A := A - N$	
	3	write A	
	4		read A
	5		$A := A + N$
	6		write A

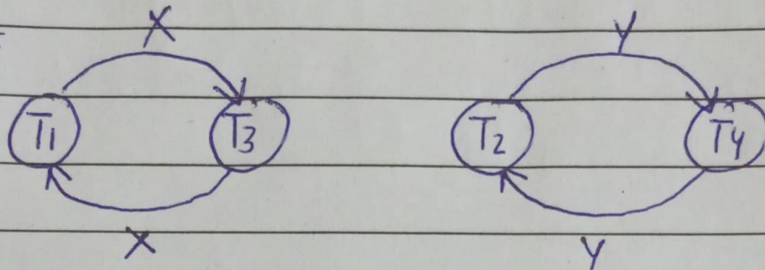
v) Real world impact

This schedule (the one in question) suffers from the Lost Update Problem. T_2 reads $A = 5000$ before T_1 writes its updated value. So T_2 computes $5000 + N$ on a stale value. When T_1 writes 4000 and then T_2 overwrites with 7000. T_1 's deduction is completely lost. In a real banking system this means a \$1000 withdrawal simply

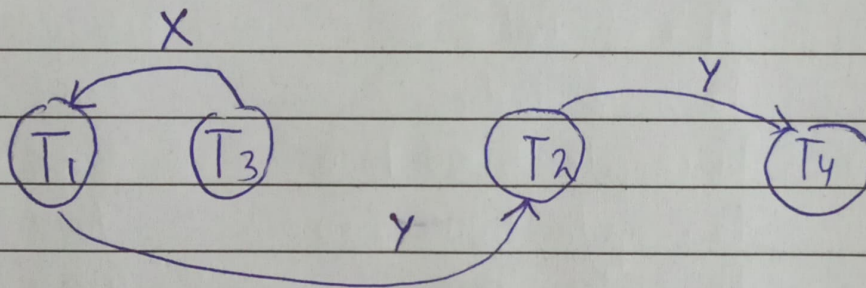
disappears from the records (money is created out of thin air causing serious financial and audit failures).

Question 4

Schedule 1



Schedule 2



ii) Serializability for Schedule 1:

As there exist two cycles in the precedence graph, the Schedule is NOT conflict serializable.

Serializability for Schedule 2:

As there exist no cycle the schedule is conflict serializable.

iii) Serial Order Schedule 1

None

Serial Order Schedule 2

$T_3 \rightarrow T_1 \rightarrow T_2 \rightarrow T_4$

Question 5

i) The problem identified is lost update. T1 and T2 both read the same initial balance (5000) before either writes back. T1's write (4000) is overwritten by T2's write (7000), so T1's deduction is completely lost.

ii)	Step	Operation	Value in DB
	1	T1 reads Balance	5000
	2	T2 reads Balance	5000
	3	T1 computes $5000 - 1000$	4000
	4	T2 computes $5000 + 2000$	7000
	5	T1 writes 4000	4000
	6	T2 writes	7000 } overwrites T1

Final Balance = 7000 (which is incorrect as T1 deduction is lost).

iii) Correct Balance if Executed Serially (if T_1 then T_2)		
Step	Operation	value in DB
1	T_1 reads Balance	5000
2	T_1 computes $5000 - 1000$	4000
3	T_1 writes	4000 commit
4	T_2 reads Balance	4000
5	T_2 computes $4000 + 2000$	6000
6	T_2 writes	6000 commit

Final Balance = 6000 (This is correct)

Q6a) Every transaction receives a unique timestamp when it starts (like a token number at a bank counter). The rule is simple which is the older transaction (lower timestamp) must appear to execute before newer ones.

Every data item (eg: x) keeps

read- $TS(x) \rightarrow$ latest timestamp that read x

write- $TS(x) \rightarrow$ latest timestamp that wrote x .

The check is that before a transaction can touch a piece of data, the system checks incoming timestamp. If it's an older transaction trying to write to data that a younger transaction has already modified, the current transaction is out of order so reject and restart transaction, otherwise we allow transaction and update read- $TS(x)$.

⇒ The problem with basic timestamp ordering

1- Cascading Restarts: A transaction that gets rejected gets a new timestamp and restarts (sent back in line) but it may conflict again and again in a starvation loop.

2- Dirty Reads: T_2 may read a value written by T_1 and if T_1 aborts, then T_2 has read dirty (invalid) data and must also abort (cascading rollback).

⇒ Strict Timestamp Ordering

This adds one extra rule that a transaction T can't read or write x until the transaction that last wrote x has committed or aborted. This prevents dirty reads and cascading roll backs.

entirely -

Example :

- T1 (TS=10) writes Balance = 4000 but hasn't committed yet.
- T2 (TS=20) wants to read Balance.
- Under basic TS: T2 is allowed to read 4000 (dirty read risk).
- Under strict TS: T2 is forced to wait until T1 commits or aborts.
- If T1 commits $\rightarrow$ T2 reads the confirmed value safely.
- If T1 aborts $\rightarrow$ T2 reads the original value - no dirty read.

This makes the system much cleaner and avoids the cascading abort problem.

Q6b) Setup

T1 (transfer $A \rightarrow B$)

- Lock (A)
- read A
- Wait for lock (B)

T2 (Transfer $B \rightarrow A$)

- Lock (B)
- read B
- Wait for lock (A)

Deadlock Arises Reason.

- T1 acquires a lock on account A and then tries to lock Account B.
- Simultaneously, T2 acquires a lock on Account B and then tries to lock account A.
- T1 is waiting for B (held by T2), and T2 is waiting for A (held by T1).
- Neither can proceed circular wait = deadlock.

Prevention Method = Lock Ordering

Assign a global order to all resources. Every transaction must always lock accounts in the same fixed order.
(eg always lock the lower account-ID first).

- Both T1 and T2 must lock A before B
- T1 locks A $\rightarrow$ tries to lock B $\rightarrow$ succeeds (T2 hasn't locked anything yet).
- T2 waits for A until T1 finishes $\rightarrow$ no circular wait $\rightarrow$ no deadlock.

Resolution Method :- Deadlock Detection + Victim Selection.

If prevention isn't used, the system can detect deadlocks using a wait-for graph. When a cycle is detected, the DBMS selects a victim transaction (usually the one that has done the least work) and aborts & restarts it, freeing the lock so the other can proceed.

In our case, T2 would be aborted releasing Lock (B), allowing T1 to complete. T2 then restarts from scratch.